

P1220R0

Controlling When Inline Functions are Emitted

Published Proposal, 2018-09-28

This version:

<http://wg21.link/P1220>

Author:

[Chris Kennelly](#) (Google)

Audience:

EWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

Provide a means to avoid emitting all inline function bodies

Table of Contents

1 Motivation

2 Proposal

3 Bikeshedding

4 Related Work

References

Informative References

§ 1. Motivation

Consider the fast allocation path for [\[gperftools\] allocation path](#):

```
template<OOMHandler>
void* malloc_fast_path(size_t size) {
    uint32 cl;
    if (PREDICT_FALSE(!Static::sizemap()->GetSizeClass(size, &cl))) {
        return tcmalloc::dispatch_allocate_full<OOMHandler>(size);
    }

    // Allocate an object from sizeclass "cl" and return elided...
}
```

With `GetSizeClass` providing a [size to size class computation](#).

Note: As of this writing, `malloc_fast_path` is structured with other checks first. This is unimportant for correctness, but does represent the most profitable opportunity for compile-time optimization.

For many allocations, we can optimize much of this lookup cost were inlining possible:

- Constant-sized allocations can have the size class fully calculated at compile-time, allowing us to jump directly to the rest of the fast path (for small sizes) or to the large allocation fallback path.
- With dynamic-sized allocations, we can partially compute the lookup based on knowledge we have at the callsite (we're allocating an array of `T`, whose size gives us guarantees that our request is a multiple of 8). For overaligned allocations, the size calculation is more complicated, but *alignment* values are frequently known at compile time (making this an even more profitable optimization).

However, we can't actually apply this to `operator new`:

- Under [dcl.inline], "an inline function...shall be defined in every translation unit in which it is odr-used." Ensuring that we can make an inlined `operator new` definition available in every translation unit may be impossible. Even with modules, the existence of any legacy, precompiled library (which almost certainly uses `operator new`) means we cannot ensure the inline function is defined in *every* translation unit.
- Under 15.5.4.6 [replacement.functions], the ability to inline the replaceable functions used for allocation is forbidden ("The program's declarations shall not be specified as inline. No

diagnostic is required."). This was the resolution of [\[LWG404\]](#). Allowing this is addressed in a separate paper [\[P1284\]](#).

This paper specifically focuses on the inlining problem that is not unique to `operator new`. Any code that has a valuable fast path but needs to avoid the overhead of a *second* function call when inlining does not happen can benefit from this technique.

Consider the parsing functions of [\[protobuf\]](#). A [fast-path is provided for parsing single-byte varints](#) with a fallback for longer values or exceptional cases (the buffer being exhausted):

```
inline bool CodedInputStream::ReadVarint32(uint32* value) {
    uint32 v = 0;
    if (PROTOBUF_PREDICT_TRUE(buffer_ < buffer_end_)) {
        v = *buffer_;
        if (v < 0x80) {
            *value = v;
            Advance(1);
            return true;
        }
    }
    int64 result = ReadVarint32Fallback(v);
    *value = static_cast<uint32>(result);
    return result >= 0;
}
```

When we fail to inline `ReadVarint32`, we are also penalized by the second function call to `ReadVarint32Fallback`, which is placed out-of-line, not in the header. (We only see `ReadVarint32` and `ReadVarint32Fallback` in one translation unit, and there's no guarantee the linker picks that definition.)

We propose a `[[noemit]]` attribute to indicate that particular definitions should *only* be used for inlining and discarded when inlining does not happen. We can make an inlineable definition available such as:

```

[[noemit]] inline bool CodedInputStream::ReadVarint32(uint32* value) {
    uint32 v = 0;
    if (PROTOBUF_PREDICT_TRUE(buffer_ < buffer_end_)) {
        v = *buffer_;
        if (v < 0x80) {
            *value = v;
            Advance(1);
            return true;
        }
    }
    int64 result = ReadVarint32Fallback(v);
    *value = static_cast<uint32>(result);
    return result >= 0;
}

```

We would then explicitly emit `ReadVarint32` in a single translation unit. Because this definition is explicitly emitted only in this one translation unit, we can arrange for it to be in the *same* translation unit as `ReadVarint32Fallback` and we can be much more aggressive in inlining the fallback code into this single, out-of-line definition. At most, the fallback code is emitted twice.

```

bool CodedInputStream::ReadVarint32(uint32*) = inline;

int64 CodedInputStream::ReadVarint32Fallback(uint32 first_byte_or_zero) {
    // ... handle remaining bytes...
}

```

§ 2. Proposal

Wording is relative to [\[N4762\]](#).

- 9.1.6 [dcl.inline]

A function declaration ([dcl.fct], [class.mfct], [class.friend]) with an `inline` specifier declares an inline function. An inline function declaration with a `[[noemit]]` attribute ([dcl.attr]) declares a *noemit inline function*. (Note: The intent is that a *noemit inline function* allows the body to be considered for inlining, but no out-of-line copy of the function would be generated in the translation unit [dcl.attr.noemit]. As `constexpr` functions are implicitly inline per [dcl.constexpr], `[[noemit]] constexpr` also declares a *noemit inline function*.)

An inline function, *except noemit inline functions*, or variable shall be defined in every translation unit in which it is odr-used. An inline function or variable ~~and~~ shall have exactly the same definition in every case ([basic.def.odr]).

- 9.11 [dcl.attr.noemit]

The attribute-token `noemit` specifies that an inline function be considered for inlining, but no out-of-line copy shall be generated in the translation unit.

- 6.2 [basic.def.odr]

Every program shall contain exactly one definition of every non-inline function or variable that is odr-used in that program outside of a discarded statement; no diagnostic required. The definition can appear explicitly in the program, it can be found in the standard or a user-defined library, or (when appropriate) it is implicitly defined (see [class.ctor], [class.dtor], [class.copy.ctor], and [class.copy.assign]). An inline function, *except noemit inline functions*, or variable shall be defined in every translation unit in which it is odr-used outside of a discarded statement. Every program shall contain exactly one explicitly-emitted definition for every *noemit inline function* that is odr-used in that program outside of a discarded statement; no diagnostic required.

- 9.4 [dcl.fct.def]

A function definition whose function-body is of the form `= inline;` is called an explicitly-emitted definition. A function that is explicitly emitted shall have a corresponding *noemit inline function body*.

§ 3. Bikeshedding

How should we invoke this feature?

- `[[noemit]] inline`, as presented
- `try inline`
- `extern inline`: Existing code may already declare functions as `extern inline`.
- `inline extern`: This sequence is less common in existing code.
- `void f() inline { ... }`

§ 4. Related Work

The approach described here has previously been used for GCC's `gnu_inline` mode ([\[gnu_inline\]](#)):

"If you specify both inline and extern in the function definition, then the definition is used only for inlining. In no case is the function compiled on its own, not even if you refer to its address explicitly. Such an address becomes an external reference, as if you had only declared the function, and had not defined it."

§ References

§ Informative References

[GNU_INLINE]

Using the GNU Compiler Collection: Inline. URL: <https://gcc.gnu.org/onlinedocs/gcc/Inline.html>

[GPERFTOOLS]

[gperftools](#). URL: <https://github.com/gperftools/gperftools>

[LWG404]

[May a replacement function be declared inline?](#). 2016-12-23. URL: <http://cplusplus.github.io/LWG/lwg-defects.html#404>

[N4762]

[Working Draft, Standard for Programming Language C++](#). 2018-07-07. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/n4762.pdf>

[P1284]

[Allowing Replaceable Functions to be Inlined](#). 2018-10-05. URL: <https://wg21.link/P1284>

[PROTOBUF]

[Protocol Buffers](#). 2018-09-22. URL: <https://github.com/protocolbuffers/protobuf>